



Simula
UiB

Algebraic Attacks on Arithmetization-Oriented Primitives - 1

Morten Øy garden

University of Bergen and Simula UiB

ALPEC Summer School, Trento

September 2026

Overview and Warm-Up



Quick Overview

Léo just mentioned the Cambrian explosion of new symmetric (Arithemization-Oriented) schemes.

This talk will focus on the subsequent cryptanalysis following this explosion.



Quick Overview

Léo just mentioned the Cambrian explosion of new symmetric (Arithemization-Oriented) schemes.

This talk will focus on the subsequent cryptanalysis following this explosion.

The oversimplified rule-of-thumb (with a lot of caveats):

- Traditional (normal) symmetric crypto: Statistical attacks **good**; Algebraic attacks **bad**.
- Arithmetization-Oriented crypto: Statistical attacks **bad**; Algebraic attacks **good**.



Recommendation

I can highly recommend Léo's [repository](#) (slides, exercises) from the spring school at FSE2025. (I've even stolen/taken inspiration in several of my own slides.)



MiMC [Alb+16]

Encryption function $F_K : \mathbb{F}_p \rightarrow \mathbb{F}_p$, for¹ $p \geq 2^\lambda$.

Let $K, C_1, \dots, C_r \in \mathbb{F}_p$ for K a secret key and C_i known round constants. The MiMC round function is

$$R_i(X) = X^3 + K + C_i, \quad 1 \leq i \leq r.$$

¹ p is also chosen so that cubing is a permutation over $\mathbb{F}_p$.



MiMC [Alb+16]

Encryption function $F_K : \mathbb{F}_p \rightarrow \mathbb{F}_p$, for¹ $p \geq 2^\lambda$.

Let $K, C_1, \dots, C_r \in \mathbb{F}_p$ for K a secret key and C_i known round constants. The MiMC round function is

$$R_i(X) = X^3 + K + C_i, \quad 1 \leq i \leq r.$$

And MiMC is given by the composition

$$F_K(X) = R_r \circ R_{r-1} \circ \dots \circ R_1(X + K)$$

¹ p is also chosen so that cubing is a permutation over $\mathbb{F}_p$.



Warm Up: Key Recovery on MiMC

For a known plaintext-ciphertext pair (P, C) , we can write the encryption function as a univariate polynomial in the variable K :

$$F_K(P) - C = G(K) \in \mathbb{F}_p[K].$$



Warm Up: Key Recovery on MiMC

For a known plaintext-ciphertext pair (P, C) , we can write the encryption function as a univariate polynomial in the variable K :

$$F_K(P) - C = G(K) \in \mathbb{F}_p[K].$$

We can find K by finding the roots of $G(K)$. For

$$D = \deg(G) = \deg(F) = 3^r,$$



Warm Up: Key Recovery on MiMC

For a known plaintext-ciphertext pair (P, C) , we can write the encryption function as a univariate polynomial in the variable K :

$$F_K(P) - C = G(K) \in \mathbb{F}_p[K].$$

We can find K by finding the roots of $G(K)$. For

$$D = \deg(G) = \deg(F) = 3^r,$$

this can be done in:

$$\mathcal{O}(D \cdot \log D \cdot \log \log D).$$

Some special cases, nuances and more concrete analysis exists [Zha+26], but this is more or less the best we can do.



Hash Function Attacks: Sponges and CICOs



Hash Functions From Permutations: The Sponge Framework

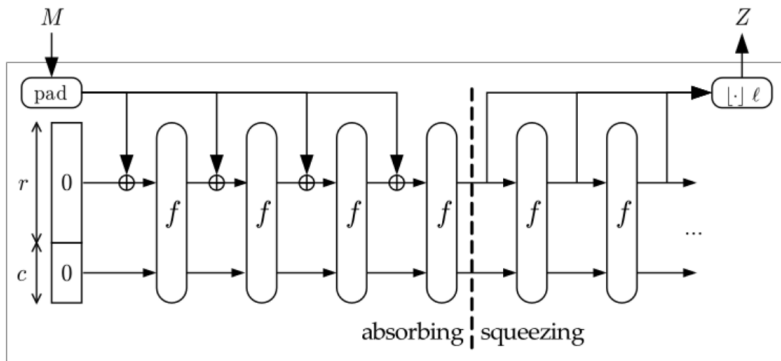


Figure: Source: Keccak.team



The CICO Problem

Constrained Input Constrained Output

The CICO problem of size c (capacity of the sponge) for a permutation P consists in finding an input/output pair such that

$$P(*, \dots, *, \underbrace{0, \dots, 0}_{c \text{ elements}}) = (*', \dots, *', \underbrace{0, \dots, 0}_{c \text{ elements}})$$



The CICO Problem

Constrained Input Constrained Output

The CICO problem of size c (capacity of the sponge) for a permutation P consists in finding an input/output pair such that

$$P(*, \dots, *, \underbrace{0, \dots, 0}_{c \text{ elements}}) = (*', \dots, *', \underbrace{0, \dots, 0}_{c \text{ elements}})$$

Solving CICO of size c *should* take about q^c operations.



The CICO-1 Problem

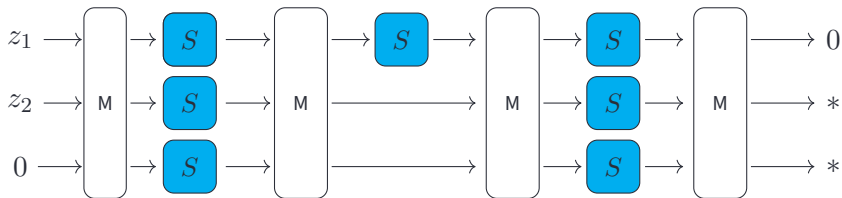
Constrained Input Constrained Output

For very large fields (e.g., 254 or 381-bit primes) it makes sense to use a capacity of 1, yielding the simplest CICO-instances.

$$P(*, \dots, *, 0) = (*', \dots, *', 0)$$



Toy Example of Poseidon2



Solving CICO-1 for Poseidon2

Idea: We can set z_2 to a random element in $\mathbb{F}_p$, and solve as a univariate polynomial for z_1 . (Similar to what we did for MiMC.)

Can we do better?



Solving CICO-1 for Poseidon2

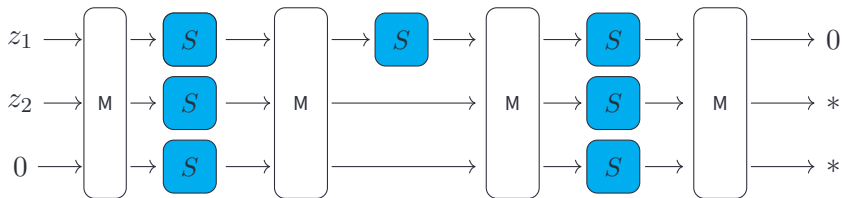
Idea: We can set z_2 to a random element in $\mathbb{F}_p$, and solve as a univariate polynomial for z_1 . (Similar to what we did for MiMC.)

Can we do better?

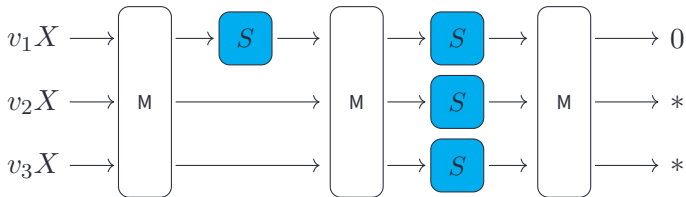
We can do a bit better, by round-skipping [Bar+22]. (Example on blackboard.)



Toy Example of Poseidon2



Poseidon2 with Round Skips



Round Skipping

More generally, round skipping refers to a family of techniques that reduces the degrees of involved polynomials.

Complements algebraic solvers very well. (As we just saw an example of.)

The techniques are very dependent on the specific schemes, matrices and CICO problem in question.

A significant amount of work on rounds-skipping has appeared in the last few months (e.g., [MG26; SV26]).



Round Skipping

More generally, round skipping refers to a family of techniques that reduces the degrees of involved polynomials.

Complements algebraic solvers very well. (As we just saw an example of.)

The techniques are very dependent on the specific schemes, matrices and CICO problem in question.

A significant amount of work on rounds-skipping has appeared in the last few months (e.g., [MG26; SV26]).

OP1: What are the optimal round skipping? Can we come up with a unifying understanding for all these new works?



CICO-1 Low Degree Schemes

For CICO-1 problems on schemes composed of low-degree polynomials, it seems unlikely that we can do better than round skips + univariate solving.



CICO-1 Low Degree Schemes

For CICO-1 problems on schemes composed of low-degree polynomials, it seems unlikely that we can do better than round skips + univariate solving.

What can we say about:

- 1) Schemes relying on other S-boxes?
- 2) CICO- k problems for $k > 1$?

We will start addressing 1) by looking at inverse S-boxes.

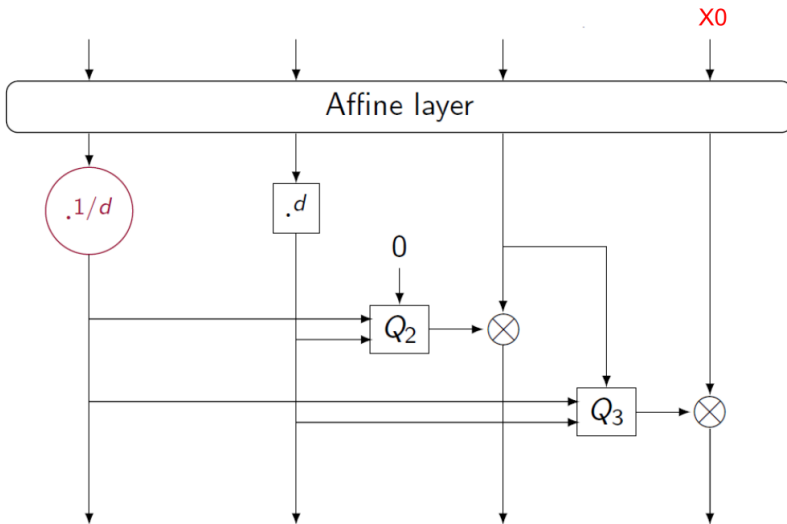


The FreeLunch Algebraic
Attack [Bar+24]
(Example with Griffin)



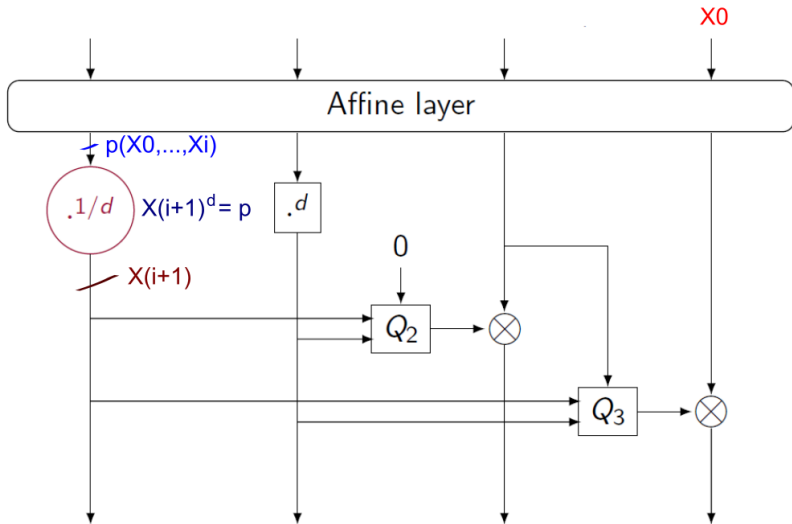
Griffin- π - Round Function (4 branches) [Gra+23]

d small (typically $d = 3$).



Griffin- π - Round Function (4 branches) [Gra+23]

d small (typically $d = 3$).



Griffin- π - Model

Let x be the original input variable and z_i the variable we introduce at round i . For $d = 3$, and r rounds the Griffin polynomial system will have the following shape:

$$\begin{aligned}z_1^3 &= f_1(x), \\z_2^3 &= f_2(x, z_1), \\&\vdots \\z_r^3 &= f_r(x, z_1, \dots, z_{r-1}), \\0 &= g(x, z_1, \dots, z_r).\end{aligned}$$

That is, a determined polynomial system of $r + 1$ equations and variables.



Recall: Solving Polynomial Systems

(From Alessio's talk yesterday)

There are some different variations and flavors, but we (roughly speaking) have three main steps for solving determined systems over large fields:



Recall: Solving Polynomial Systems

(From Alessio's talk yesterday)

There are some different variations and flavors, but we (roughly speaking) have three main steps for solving determined systems over large fields:

- 1) Compute a Gröbner basis.



Recall: Solving Polynomial Systems

(From Alessio's talk yesterday)

There are some different variations and flavors, but we (roughly speaking) have three main steps for solving determined systems over large fields:

- 1) Compute a Gröbner basis.
- 2) FGLM-type step: compute multiplication matrix and find a univariate polynomial $F(x) \in I$.



Recall: Solving Polynomial Systems

(From Alessio's talk yesterday)

There are some different variations and flavors, but we (roughly speaking) have three main steps for solving determined systems over large fields:

- 1) Compute a Gröbner basis.
- 2) FGLM-type step: compute multiplication matrix and find a univariate polynomial $F(x) \in I$.
- 3) Wrap up: Use F to find a valid solution for x . Recover the remaining variables.



Recall: Solving Polynomial Systems

(From Alessio's talk yesterday)

There are some different variations and flavors, but we (roughly speaking) have three main steps for solving determined systems over large fields:

- 1) Compute a Gröbner basis.
- 2) FGLM-type step: compute multiplication matrix and find a univariate polynomial $F(x_0) \in I$.
- 3) Wrap up: Use F to find a valid solution for x_0 . Recover the remaining variables.

Steps 1) and 2) are usually expensive; step 3) comparatively easy.



Recall: Solving Polynomial Systems

(From Alessio's talk yesterday)

There are some different variations and flavors, but we (roughly speaking) have three main steps for solving determined systems over large fields:

- 1) **Compute a Gröbner basis.**
- 2) **FGLM-type step: compute multiplication matrix and find a univariate polynomial $F(x_0) \in I$.**
- 3) **Wrap up: Use F to find a valid solution for x_0 . Recover the remaining variables.**

Steps 1) and 2) are usually expensive; step 3) comparatively easy.

Usual approach (e.g., from PQ-crypto): try to estimate the hardness of 1) and rely on this. This was initially done by the authors of several schemes.



Computing Gröbner Bases in Our Setting

Our polynomials systems are very structured (as we just saw with Griffin). Instead of the usual approach, we will rely on this result:

Proposition

A set of polynomials $G = \{g_1, \dots, g_\ell\}$ is a Gröbner basis for $I = \langle G \rangle$ if

$$\text{LM}_{<}(g_1), \dots, \text{LM}_{<}(g_\ell)$$

are pairwise **coprime**.

(E.g. x^2 and y are coprime; x^2 and xy are not.)



Polynomials from Griffin

Back to Griffin, we had polynomials on this form:

$$z_1^3 = f_1(x),$$

$$z_2^3 = f_2(x, z_1),$$

$$\vdots$$

$$z_r^3 = f_r(x, z_1, \dots, z_{r-1}),$$

$$0 = g(x, z_1, \dots, z_r).$$



Griffin, $r = 2$

For $d = 3$ and $r = 2$ it looks something like this:

$$\underline{z_1^3} = ax + b$$

$$\underline{z_2^3} = \underline{cx^7} + \dots$$

$$\underline{x^{49}} + dx^{46} + ex^{45} + \dots = 0$$



Griffin, $r = 2$

For $d = 3$ and $r = 2$ it looks something like this:

$$\underline{z_1^3} = ax + b$$

$$\underline{z_2^3} = \underline{cx^7} + \dots$$

$$\underline{x^{49}} + dx^{46} + ex^{45} + \dots = 0$$

In **grevlex** (degree-first), the leading monomials are $\underline{z_1^3}$, $\underline{x^7}$ and $\underline{x^{49}}$.

$\implies$ The Proposition does not apply.



Griffin, $r = 2$

For $d = 3$ and $r = 2$ it looks something like this:

$$\underline{z_1^3} = ax + b$$

$$\underline{z_2^3} = \underline{cx^7} + \dots$$

$$\underline{x^{49}} + dx^{46} + ex^{45} + \dots = 0$$

In **grevlex** (degree-first), the leading monomials are $\underline{z_1^3}$, x^7 and x^{49} .

$\implies$ The Proposition does not apply.

But we are very close here. Can we fix this by changing the monomial order?



Monomial Order and Weight

Recall from Alessio's talk: there exists a (unique) reduced Gröbner basis for any monomial order.



Monomial Order and Weight

Recall from Alessio's talk: there exists a (unique) reduced Gröbner basis for any monomial order.

All monomial orders can be thought of through weight vectors $(\text{wt}(x_0), \dots, \text{wt}(x_{n-1}))$, where monomials are compared by the values

$$x_0^{a_0} \cdots x_{n-1}^{a_{n-1}} \longrightarrow a_0 \text{wt}(x_0) + \dots + a_{n-1} \text{wt}(x_{n-1}).$$



Monomial Order and Weight

Recall from Alessio's talk: there exists a (unique) reduced Gröbner basis for any monomial order.

All monomial orders can be thought of through weight vectors $(\text{wt}(x_0), \dots, \text{wt}(x_{n-1}))$, where monomials are compared by the values

$$x_0^{a_0} \cdots x_{n-1}^{a_{n-1}} \longrightarrow a_0 \text{wt}(x_0) + \dots + a_{n-1} \text{wt}(x_{n-1}).$$

Example: Grading (comparison by degree) has weight vector

$$\text{wt}(x_0) = \dots = \text{wt}(x_{n-1}) = 1.$$

Note that grading is only one choice of (infinitely) many for monomial orders.



Griffin $r = 2$ again

$$\underline{z_1^3} = ax + b$$

$$\underline{z_2^3} = cx^7 + \dots$$

$$\underline{x^{49}} + dx^{46} + ex^{45} + \dots = 0$$



Griffin $r = 2$ again

$$\underline{z_1^3} = ax + b$$

$$\underline{z_2^3} = cx^7 + \dots$$

$$\underline{x^{49}} + dx^{46} + ex^{45} + \dots = 0$$

In an order with $\text{wt}(x) = \text{wt}(z_1) = 1$ and $\text{wt}(z_2) = 3$, the leading monomials are $\underline{z_1^3}$, $\underline{z_2^3}$ and $\underline{x^{49}}$.

$\implies$ It's a Gröbner basis!



CICO-1 for Griffin

This choice of monomial order generalizes to any number of rounds.

The complexity overview for CICO-1 problems for Griffin looks something like this:

- 1) Compute a Gröbner basis.
- 2) FGLM-type step: compute multiplication matrix and find a univariate polynomial $F(x_0) \in I$.
- 3) Wrap up: Use F to find a valid solution for x_0 . Recover the remaining variables.

Step 2) is expensive; steps 1) and 3) comparatively easy.



Complexity of Step 2)

Caveats for the pedant

Technical Note I: I'm being very cavalier in what I refer to as “FGLM-variant” here. That's fine for us: it all boils down to linear algebra over a space of size D_I .

Technical Note II: I'll assume that a multiplication matrix can be easily computed from our GB. That's typically the case in practice, but we don't have general results for this.



Complexity of Step 2)

Caveats for the pedant

Technical Note I: I'm being very cavalier in what I refer to as "FGLM-variant" here. That's fine for us: it all boils down to linear algebra over a space of size D_I .

Technical Note II: I'll assume that a multiplication matrix can be easily computed from our GB. That's typically the case in practice, but we don't have general results for this.

OP2: There's currently a lot of ad-hoc approaches to constructing the multiplication matrix. Can we get more general results in our setting?



Complexity of Step 2)

Recall the *degree of the ideal* I , D_I , which is defined by:

$$D_I = \dim_{\mathbb{F}_p} (\mathbb{F}_p[x_1, \dots, x_n]/I) .$$



Complexity of Step 2)

Recall the *degree of the ideal* I , D_I , which is defined by:

$$D_I = \dim_{\mathbb{F}_p} (\mathbb{F}_p[x_1, \dots, x_n]/I) .$$

The cost of step 2) essentially boils down to linear algebra over a space of size D_I . This is done in

$$\mathcal{O}(D_I^\omega)$$

$\mathbb{F}_p$ -operations, where ω is the linear algebra constant.



Complexity of Step 2)

$$\text{Attack Complexity} = \mathcal{O}(D_I^\omega)$$

What is ω ?

- **In practice:** $\omega \approx 2.81$. (Strassen's algorithm).
- **Asymptotically:** $\omega \approx 2.37$ (Coppersmith-Winograd type galactic algorithm).
- **For the paranoid working cryptographer:** $\omega = 2$. (Lower bounds Wiedemann-type solvers that exploits sparsity.)



Complexity of Step 2)

$$\text{Attack Complexity} = \mathcal{O}(D_I^\omega)$$

What is ω ?

- **In practice:** $\omega \approx 2.81$. (Strassen's algorithm).
- **Asymptotically:** $\omega \approx 2.37$ (Coppersmith-Winograd type galactic algorithm).
- **For the paranoid working cryptographer:** $\omega = 2$. (Lower bounds Wiedemann-type solvers that exploits sparsity.)

D_I is something that we can easily derive once we have a Gröbner basis. (**Griffin example on blackboard.**)



Experimental Results

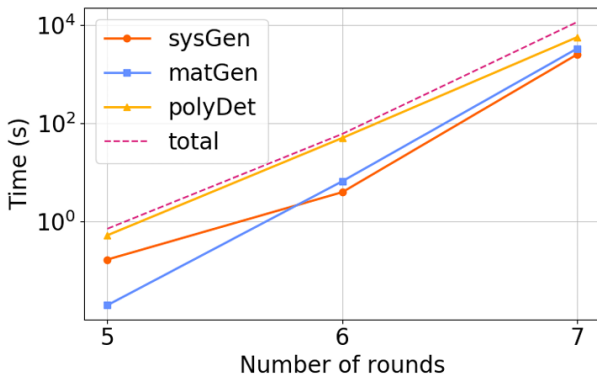


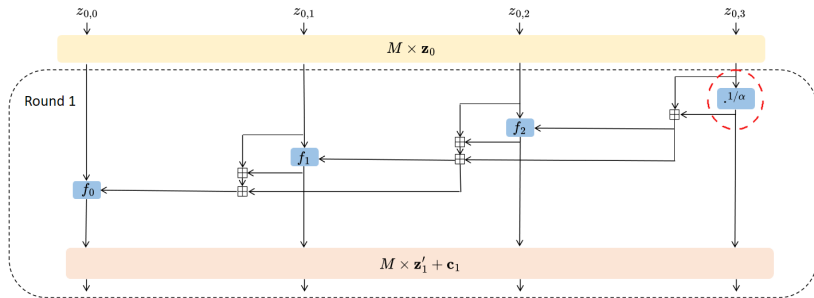
Figure: Experimental timings for Griffin (7 out of 10 rounds, $d=3$).



Further Examples and Impact



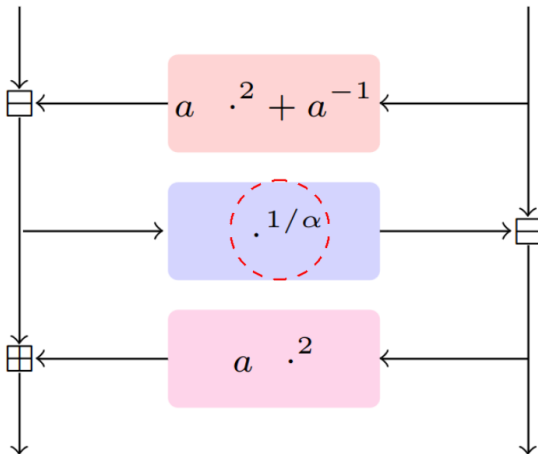
Arion- π - Round Function (4 branches) [RST23]



$\cdot 1/\alpha$ is the only high-degree operation $\implies$ add one variable per $\cdot 1/\alpha$.



Anemoi [Bou+23] - Nonlinear Function



Some Nuances on Anemoi

The techniques we just saw doesn't apply directly to Anemoi (for technical reasons).

Instead, we can tweak it by essentially going into a larger ideal. That works, but it's not optimal: larger D_I .



Some Nuances on Anemoi

The techniques we just saw doesn't apply directly to Anemoi (for technical reasons).

Instead, we can tweak it by essentially going into a larger ideal. That works, but it's not optimal: larger D_I .

A different, more bespoke analysis for Anemoi [CR25] avoids this problem and manages to achieve this optimality w.r.t. D_I .



Impact of Attacks

Breaks some CICO-1 variants of Griffin, Arion and Anemoi.

Other parameters survive, and we can increase the number of rounds to avoid these attacks.



Impact of Attacks

Breaks some CICO-1 variants of Griffin, Arion and Anemoi.

Other parameters survive, and we can increase the number of rounds to avoid these attacks.

At a higher level: we now know more about D_I and how to use it in security arguments.



Quick Recap

So far we've mainly been looking at ZK-friendly hash functions.

Setting (so far) is CICO-1, where we can imagine huge fields.

After lunch: More techniques (resultants) + more general CICO problems.



References I

- [Alb+16] Martin Albrecht et al. “MiMC: Efficient encryption and cryptographic hashing with minimal multiplicative complexity”. In: *International Conference on the Theory and Application of Cryptology and Information Security*. Springer. 2016, pp. 191–219.
- [Bar+22] Augustin Bariant et al. “Algebraic attacks against some arithmetization-oriented primitives”. In: *IACR Transactions on Symmetric Cryptology* (2022), pp. 73–101.



References II

- [Bar+24] Augustin Bariant et al. “The Algebraic FreeLunch: Efficient Gröbner Basis Attacks Against Arithmetization-Oriented Primitives”. In: *Annual International Cryptology Conference*. Springer. 2024, pp. 139–173.
- [Bou+23] Clémence Bouvier et al. “New design techniques for efficient arithmetization-oriented hash functions: Anemoi permutations and Jive compression mode”. In: *Annual International Cryptology Conference*. Springer. 2023, pp. 507–539.



References III

- [CR25] Luca Campa and Arnab Roy. “Groebner basis cryptanalysis of anemoi”. In: *Annual International Conference on the Theory and Applications of Cryptographic Techniques*. Springer. 2025, pp. 303–332.
- [Gra+23] Lorenzo Grassi et al. “Horst meets fluid-spn: Griffin for zero-knowledge applications”. In: *Annual International Cryptology Conference*. Springer. 2023, pp. 573–606.
- [MG26] Simon-Philipp Merz and Àlex Rodríguez García. “Skipping Class: Algebraic Attacks exploiting weak matrices and operation modes of Poseidon2 (b)”. In: *Cryptology ePrint Archive* (2026).



References IV

- [RST23] Arnab Roy, Matthias Johann Steiner, and Stefano Trevisani. “Arion: arithmetization-oriented permutation and hashing from generalized triangular dynamical systems”. In: *arXiv preprint arXiv:2303.04639* (2023).
- [SV26] Antonio Sanso and Giuseppe Vitto. “Top Gun: Degree Annihilation Attacks on Poseidon”. In: *Cryptology ePrint Archive* (2026).
- [Zha+26] Ziyu Zhao et al. “Graeffe-based attacks on Poseidon and NTT lower bounds”. In: *Annual International Cryptology Conference*. Springer. 2026, pp. 471–498.

